

Appendix

Details for Benchmark Datasets

The UCR (Dau et al. 2019) (University of California, Riverside) Time Series Archive is a comprehensive collection of time series datasets that supports research and development across a wide range of time series analysis domains. Curated by researchers at UCR, the archive features datasets from diverse fields such as biology, finance, healthcare, environmental science, and beyond. These datasets serve as valuable resources for exploring and developing algorithms aimed at tasks including classification, clustering, anomaly detection, forecasting, and other forms of time series analysis.

The UCR Time Series Archive is widely recognized within the research community due to its well-organized structure, standardized formats, and the availability of both univariate and multivariate time series. It has become a benchmark for comparing various time series analysis techniques, enabling the evaluation of model performance. The archive’s diversity—spanning variations in data characteristics such as series length, noise levels, and domain-specific features—makes it an essential resource for addressing a broad spectrum of time series problems, extending beyond just classification tasks. The specific dataset information is shown in Table 1.

Details on Baselines

To comprehensively evaluate the performance of TFMCC, this paper selects 6 representative algorithms as baseline methods for comparative experiments. They are as follows:

STCN(Ma et al. 2020): A self-supervised time series clustering network that integrates self-supervised learning with clustering tasks, ensuring both representation effectiveness and clustering accuracy.

TCGAN(Huang and Deng 2023): A generative adversarial framework for time series that uses adversarial training between the generator and discriminator to learn representations of time series data, thereby enhancing clustering performance.

TS2Vec(Yue et al. 2022): An unsupervised model for general time series representation learning, which employs hierarchical time contrastive learning to extract multi-scale features, excelling in tasks such as regression, classification, and prediction. For clustering tasks, FCM is applied.

TimesURL(Liu and Chen 2024): A time series representation learning framework that incorporates time-domain and frequency-domain enhancement strategies. It introduces a hard negative sample generation mechanism and achieves superior performance in downstream tasks such as prediction, classification, and anomaly detection by optimizing contrastive learning and reconstruction. We apply FCM for clustering tasks.

CDCC(Peng et al. 2024): A time series clustering framework that integrates contrastive learning with both time and frequency domain information. It uses a dual-view autoencoder for representation learning and merges instance-level and cluster-level contrastive learning to enhance clustering-friendly optimization.

R-Clustering(Jorge and Rubén 2024): An efficient time series clustering method that utilizes random convolution kernels and principal component analysis (PCA). It combines random feature extraction and dimensionality reduction with K-means, achieving superior clustering performance and scalability.

The Impact of Batchsize

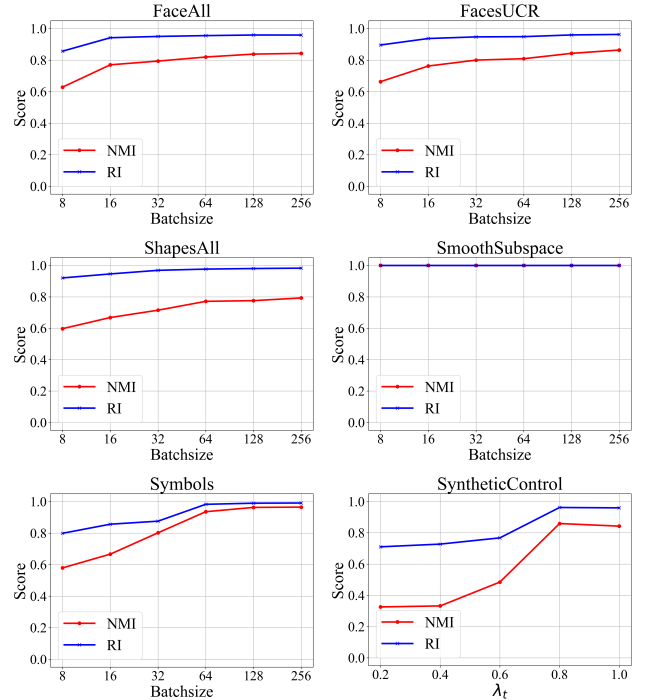


Figure 1: The impact of batchsize

In deep learning, batch size is an important factor that affects the model’s training process and final performance. To evaluate the impact of batch size, we trained TFMCC on six datasets: FaceAll, FacesUCR, SmoothSubspace, Symbols, SyntheticControl, and ShapesAll. We used batch sizes in the range of 8, 16, 32, 64, 128, 256 for the experiments. We found that batch size had a significant impact on clustering performance, particularly on Normalized Mutual Information (NMI) and Rand Index (RI). The experimental results show that as the batch size increases, the model’s stability and performance in feature learning improve. This is consistent with the findings in existing literature (Zhong et al. 2020; Li et al. 2021), which suggests that larger batch sizes can enhance model performance and generalization ability by providing more diverse samples in each training step. Specifically, as the batch size increased from 8 to 256, both NMI and RI scores steadily improved, indicating a significant enhancement in clustering performance.

However, we also observed that although larger batch sizes improve model performance, they come with higher computational costs and memory requirements. As the batch size increases, the computational resources needed for each training step also rise. Moreover, we observed that after

Name	Sample	Class	Length	Name	Sample	Class	Length
AllGestureWiimoteX	1000	10	Vary	Lightning7	143	7	319
AllGestureWiimoteY	1000	10	Vary	Meat	120	3	448
AllGestureWiimoteZ	1000	10	Vary	MelbournePedestrian	3633	10	24
Beef	60	5	470	MiddlePhalanxOutlineAgeGroup	554	3	80
Car	120	4	577	MoteStrain	1272	2	84
CBF	930	3	128	OSULeaf	442	6	427
CricketX	780	12	300	PickupGestureWiimoteZ	100	10	Vary
CricketY	780	12	300	ProximalPhalanxTW	605	6	80
CricketZ	780	12	300	ShakeGestureWiimoteZ	100	10	Vary
DodgerLoopDay	158	7	288	ShapeletSim	200	2	500
ECGFiveDays	884	2	136	ShapesAll	1200	60	512
FaceAll	2250	14	131	SmoothSubspace	300	3	15
FaceFour	112	4	350	SonyAIBORobotSurface1	621	2	70
FacesUCR	2250	14	131	SonyAIBORobotSurface2	980	2	65
FiftyWords	905	50	270	SwedishLeaf	1125	15	128
FordB	4446	2	500	Symbols	1020	6	398
GestureMidAirD3	338	26	Vary	SyntheticControl	600	6	60
GesturePebbleZ1	304	6	Vary	ToeSegmentation2	166	2	343
GesturePebbleZ2	304	6	Vary	TwoPatterns	5000	4	128
GunPointOldVersusYoung	451	2	150	WordSynonyms	905	25	270

Table 1: Dataset and their data descriptions

a certain threshold, the performance improvement begins to slow down, indicating that beyond a certain batch size, the marginal gains in performance become less significant. Therefore, there exists an optimal batch size that strikes a balance between computational efficiency and model performance. Based on our experimental results, we recommend selecting a batch size between 64 and 256 to achieve the best clustering performance without incurring excessive computational overhead. In this experiment, we used a batch size of 256.

Significance Test

We also conduct pairwise comparisons between TFMCC and other method using Wilcoxon signed-rank tests. The results of the significance tests are presented in the last row (P-Value) of Table 2. At a significance level of $p < 0.05$, TFMCC shows significant superiority over all the compared methods.

Visualization

To further validate the performance and optimization of the model, we analyze the representation distribution of the samples. Figure 2 shows the t-SNE visualization effect of the changes in the distribution of SmoothSubspace (two subplots on the left) and SyntheticControl (two subplots on the right) datasets before and after training. Before training, the sample distribution is scattered and the clustering boundaries are not clear. After training, the samples become more compact and form distinct clusters with increased intra-cluster cohesion and improved inter-cluster boundaries. This indicates that the model effectively improves the discriminative ability of feature representation and clustering performance.

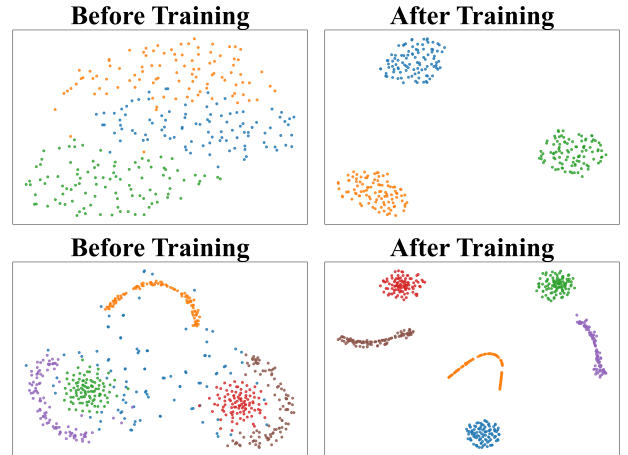


Figure 2: Comparison of sample representation distributions using t-SNE.

At the request of the reviewers, to further validate the model’s performance and optimisation effects, we additionally employed UMAP to analyse the distribution of sample representations. Figure 3 presents the UMAP visualisation of distribution changes before and after training for the SmoothSubspace (left two subfigures) and SyntheticControl (right two subfigures) datasets. The UMAP results similarly demonstrate: prior to training, the sample distribution appeared scattered with indistinct clustering boundaries. Following training, samples coalesce into more compact clusters with enhanced cohesion within clusters and clearer boundaries between clusters. This further validates that the model effectively enhances the discriminative capa-

Method	STCN	TCGAN	CDDC	TimesURL	TS2Vec	R-Clust
NMI P-value	2e-12	1e-08	6e-08	2e-12	2e-06	8e-06
RI P-value	9e-12	7e-09	3e-07	8e-11	1e-05	2e-05

Table 2: Comparison of P-values for Different Methods

bility and clustering performance of feature representations.

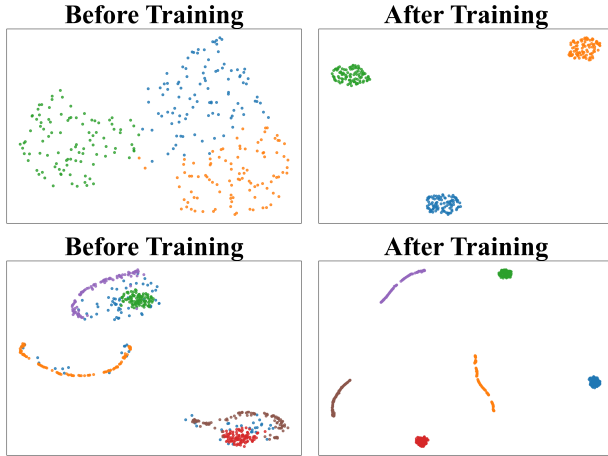


Figure 3: Comparison of sample representation distributions using UMAP.

The Impact of Loss Hyperparameters

At the reviewer’s request, to further validate the model’s performance, we additionally introduced learning weights into the loss component to observe whether performance would be enhanced.

As shown in Figure 4, we conducted an analysis of the impact of loss hyperparameters (time-level contrastive loss λ_t , instance-level contrastive loss λ_I , and cluster-level contrastive loss λ_C). The experimental results indicate significant differences in performance across different datasets, with the sensitivity to these hyperparameters varying for each dataset. Specifically, the optimal values for λ_t , λ_I , and λ_C differ depending on the dataset. Although adjustments to these hyperparameters result in some performance fluctuations on certain datasets, the overall trend suggests that the impact of these hyperparameters on model performance is not consistent.

Therefore, to achieve better performance in different datasets, we recommend introducing learnable weights for each loss level (time-level λ_t , instance-level λ_I , and cluster-level λ_C) to more precisely optimize the contribution of each contrastive loss level, thereby enhancing clustering effectiveness.

Running Time Comparison

Although the computational cost for time series analysis is generally lower compared to tasks like image or video processing, the multi-scale augmentation strategy and multi-

level contrastive learning mechanism do introduce additional computational overhead. To assess the complexity of our model, we compared the average runtime of each method across 40 UCR datasets, with all methods using the same number of epochs and other experimental parameters. The average results are shown in Table 3.

References

- Dau, H. A.; Bagnall, A.; Kamgar, K.; Yeh, C.-C. M.; Zhu, Y.; Gharghabi, S.; Ratanamahatana, C. A.; and Keogh, E. 2019. The UCR time series archive. *IEEE/CAA Journal of Automatica Sinica*, 6(6): 1293–1305.
- Huang, F.; and Deng, Y. 2023. TCGAN: Convolutional Generative Adversarial Network for time series classification and clustering. *Neural Networks*, 165: 868–883.
- Jorge, M.-B.; and Rubén, C. 2024. Time series clustering with random convolutional kernels. *Data Mining and Knowledge Discovery*, 1–27.
- Li, Y.; Hu, P.; Liu, Z.; Peng, D.; Zhou, J. T.; and Peng, X. 2021. Contrastive clustering. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, 8547–8555.
- Liu, J.; and Chen, S. 2024. Timesurl: Self-supervised contrastive learning for universal time series representation learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 13918–13926.
- Ma, Q.; Li, S.; Zhuang, W.; Wang, J.; and Zeng, D. 2020. Self-supervised time series clustering with model-based dynamics. *IEEE Transactions on Neural Networks and Learning Systems*, 32(9): 3942–3955.
- Peng, F.; Luo, J.; Lu, X.; Wang, S.; and Li, F. 2024. Cross-Domain Contrastive Learning for Time Series Clustering. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 8921–8929.
- Yue, Z.; Wang, Y.; Duan, J.; Yang, T.; Huang, C.; Tong, Y.; and Xu, B. 2022. Ts2vec: Towards universal representation of time series. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, 8980–8987.
- Zhong, H.; Chen, C.; Jin, Z.; and Hua, X.-S. 2020. Deep robust clustering by contrastive learning. *arXiv preprint arXiv:2008.03030*.

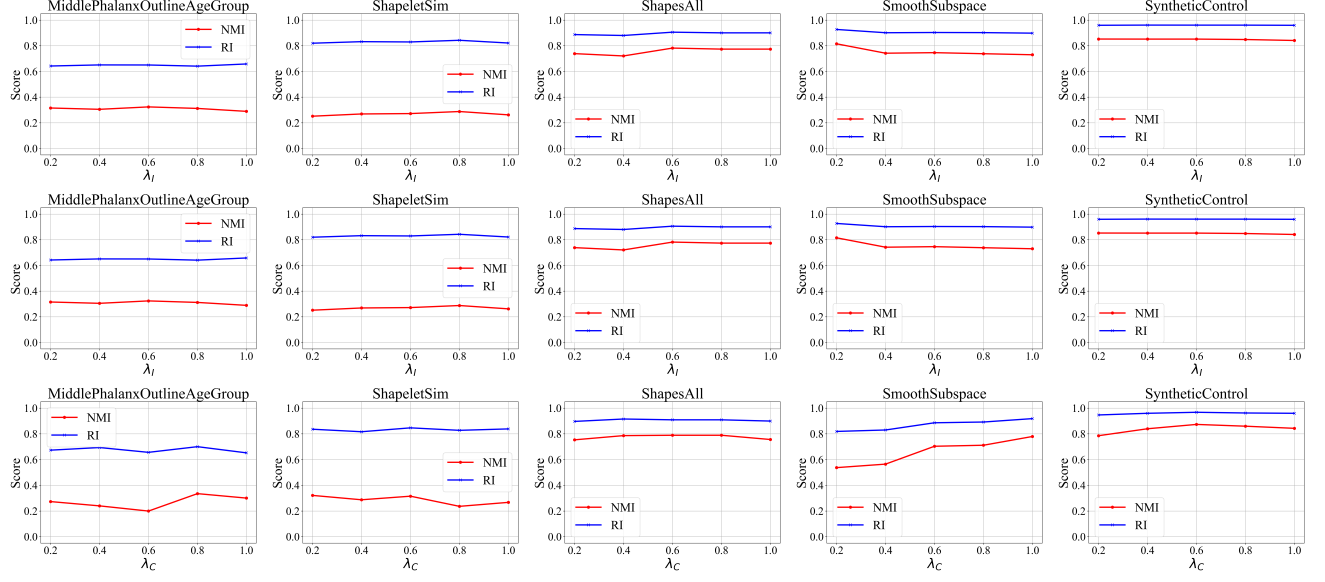


Figure 4: Impact of Loss Hyperparameters (λ_t , λ_I , and λ_C) on Clustering Performance Across Datasets.

Running Time(s)	STCN	TCGAN	CDCC	TimesURL	TS2Vec	R-Clust	TFMCC
AVG Time(s)	133	91	42	748	26	6	37

Table 3: Running Time Comparison Across Different Methods on 40 UCR Datasets